МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Языки программирования

Контейнеры и алгоритмы STL

Руководитель _____ А.В. Проскурин
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

Получение практических навыков разработки и отладки программ по обработке строк с использованием стандартной библиотеки шаблонов (STL).

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с общей постановкой задачи.

2. Ознакомится с вариантом задания – соответствует вашему номеру в списке группы (при нехватке заданий вариант задания вычисляется как номер_в_списке_группы - количество_заданий).

3. Разработать класс согласно варианту задания.

4. Написать и отладить программу на подготовленных наборах тестовых данных.

5. Подготовить отчет по лабораторной работе. Отчет должен включать в себя:

• титульный лист;

• цель лабораторной работы;

• постановку задачи;

• текст программы с комментариями;

• демонстрацию работы программы (Снимки экрана при выполнении действий программы с описанием).

• краткие ответы на контрольные вопросы;

• выводы по лабораторной работе.

6. Защитить лабораторную работу перед преподавателем.

ПОСТАНОВКА ЗАДАЧИ

Необходимо реализовать класс (а также при необходимости шаблонные функции), который будет выполнять обработку текстового файла согласно варианту задания.

При реализации необходимо использовать контейнеры и алгоритмы стандартной библиотеки шаблонов (STL) подходящие для решения задачи (например, контейнеры: vector, map, set, list, и алгоритмы: find, sort, и т.п.), а также методы класса string.

В зависимости от оценки, на которую вы претендуете, необходимо выполнить следующие задания (Для каждой следующей оценки нужно выполнить ВСЕ предыдущие задания, если обратное не указано явно):

Оценка «**Удовлетворительно**»:

1. Написать класс, выполняющий обработку текста согласно варианту

Оценка «**Хорошо**»:

2. Реализовать работу с текстовым файлом: исходные данные берутся из одного файла, результат записывается в другой.

Оценка «**Отлично**»:

3. С помощью функций реализовать меню для организации взаимодействия с пользователем. С помощью меню необходимо выполнить демонстрацию функционирования всей программы (т.е. после выполнения действия происходит вывод меню до тех пор, пока в меню не будет выбран пункт «Закончить работу с программой»).

Вариант №5.

Формулировка задания: На основе исходного текста, содержащего не менее 10 предложений, необходимо найти и вывести на экран какая английская буква встречается в каждом предложении чаще всего и сколько раз. Если таких букв несколько, то программа должна выводить на экран ту из них, которая стоит по алфавиту раньше.

Формат входных данных: произвольный текст, содержащий не менее 10 строк, в каждой строке не более 80 символов. В качестве разделителей предложений выступают знаки: ‘.’, ‘!’, ‘?’.

Пример: файл содержит следующую информацию (фрагмент для демонстрации):

It is not a simple task. Yes!

В первом предложении чаще всего встречаются буквы I, S и T, следовательно, программа для первого предложения должна вывести:

Предложение №1: I 3.

ХОД РАБОТЫ

Файл **TextProcessor.h**:

```cpp
#ifndef TEXTPROCESSOR_H
#define TEXTPROCESSOR_H

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <map>
#include <algorithm>
#include <cctype>
#include <sstream>
#include <set>
#include <iomanip>

using namespace std;

/// @brief Класс для обработки текста
class TextProcessor {
private:
    string text; // Исходный текст для последующей обработки
    vector<string> sentences; // Массив извлечённых предложений из текста
    const string delimiters = ".!?"; // Разделители предложений

public:
    /// @brief Конструктор по умолчанию. Создаёт объект с пустыми полями
    TextProcessor() = default;

    /// @brief Конструктор с параметрами. Принимает строку текста и сохраняет её в виде предложений
    /// @param t строка текста
    TextProcessor(const string& t) : text(t) {
        splitIntoSentences();
    }

    /// @brief Загрузка текста из файла
    /// @param filename название файла
    void loadFromFile(const string& filename) {
        ifstream file(filename);
        if (!file.is_open()) {
            cout << "ERROR: Cannot open file " << filename << endl;
            return;
        }

        text.clear();
        string line;
        while (getline(file, line)) {
            text += line + " ";
        }
        file.close();

        // Убираем лишние пробелы в начале и конце
        text.erase(0, text.find_first_not_of(" \t\n\r"));
        text.erase(text.find_last_not_of(" \t\n\r") + 1);

        cout << "Text loaded from " << filename << endl;
        splitIntoSentences();
    }

    /// @brief Сохранение текста в файл
    /// @param filename название файла
    void saveToFile(const string& filename) const {
        ofstream file(filename);
        if (!file.is_open()) {
```

```
62          if (!file.is_open()) {
63              cout << "ERROR: Cannot open file " << filename << endl;
64              return;
65          }
66
67          file << text;
68          file.close();
69          cout << "Text saved to " << filename << endl;
70      }
71
72      /// @brief Сеттер для текста в процессоре. Принимает строку и разбивает её на предложения
73      /// @param t новый текст для добавления в процессор
74      void setText(const string& t) {
75          text = t;
76          splitIntoSentences();
77      }
78
79      /// @brief Геттер для текста в процессоре
80      /// @return указатель на строку с текстом
81      const string& getText() const { return text; }
82
83      /// @brief Геттер для предложений в процессоре
84      /// @return указатель на массив предложений
85      const vector<string>& getSentences() const { return sentences; }
86
87      /// @brief Поиск и вывод самых частых символов в каждом предложении в процессоре
88      void findMostFrequentLetterInEachSentence() const {
89          if (sentences.empty()) {
90              cout << "ERROR: No sentences found in the text." << endl;
91              return;
92          }
93          // Для каждого предложения получаем набор символов и находим самый частый из них
94          for (size_t i = 0; i < sentences.size(); ++i) {
95              string letters = extractLettersOnly(sentences[i]);
96              auto [letter, count] = findMostFrequentLetter(letters);
97
98              if (letter != '\0') {
99                  cout << "Sentence #" << (i + 1) << ": " << static_cast<char>(toupper(letter)) << " " << count << "." << endl;
100             } else {
101                 cout << "Sentence #" << (i + 1) << ": No letters found." << endl;
102             }
103         }
104     }
105
106     /// @brief Разделение текста в процессоре на предложения по символам-разделителям
107     void splitIntoSentences() {
108         sentences.clear();
109         string current_sentence; // Текущее обрабатываемое предложение
110         for (char c : text) {
111             if (delimiters.find(c) != string::npos) { // Символ строки является разделителем?
112                 if (!current_sentence.empty()) { // Предложение закончилось, добавляем в массив
113                     sentences.push_back(current_sentence);
114                     current_sentence.clear();
115                 }
116             } else { // Символ строки не является разделителем?
117                 current_sentence += c; // Добавляем его к текущему предложению
118             }
119         }
```

```
        }
        // Добавляем последнее предложение, если оно не пустое
        if (!current_sentence.empty()) {
            sentences.push_back(current_sentence);
        }
    }

    /// @brief Извлечение из строки только букв, игнорируя другие символы
    /// @param sentence строка текста
    /// @return строка, содержащая извлечённые буквы
    string extractLettersOnly(const string& sentence) const {
        string letters; // Строка для хранения букв
        for (char c : sentence) { // Проходимся по каждому символу в строке
            if (isalpha(c)) { // Является ли латинской буквой?
                letters += tolower(c); // Преобразуем в нижний регистр и плюсуем в строку с буквами
            }
        }
        return letters;
    }

    /// @brief Поиск самого частого элемента в строке из букв
    /// @param letters строка с буквами
    /// @return кортеж: буква + количество её вхождений
    pair<char, int> findMostFrequentLetter(const string& letters) const {
        if (letters.empty()) {
            return {'\0', 0};
        }

        map<char, int> freq; // Ассоциативный список (буква: кол-во вхождений)
        for (char c : letters) { // Проход по каждой букве в строке
            freq[c]++; // Увеличение счётчика для отдельной буквы
        }

        int max_freq = 0; // Максимальная частота символа
        for (const auto& p : freq) { // Проход по каждой позиции ассоц. списка
            if (p.second > max_freq) { // Ищем символ с максимумом вхождений
                max_freq = p.second;
            }
        }

        char most_frequent_letter = 'z' + 1; // Самая большая латинская строчная буква
        for (const auto& p : freq) {
            if (p.second == max_freq && p.first < most_frequent_letter) { // Ищем наименьшую букву
                most_frequent_letter = p.first;
            }
        }

        return {most_frequent_letter, max_freq};
    }

    /// @brief Вывод текста их процессора
    void displayText() const {
        cout << "\n--- Original Text ---" << endl;
        cout << text << endl;
    }
```

```
174
175     /// @brief Вывод массива предложений из процессора
176     void displaySentences() const {
177         cout << "\n--- Sentences ---" << endl;
178         for (size_t i = 0; i < sentences.size(); ++i) {
179             cout << "#" << (i + 1) << ": " << sentences[i] << endl;
180         }
181     }
182 };
183
184 #endif
```

Файл **main.cpp**:

```cpp
#include "TextProcessor.h"
#include <iostream>
using namespace std;

int main() {
    TextProcessor processor;
    bool running = true;

    while (running) {
        cout << "\n=====================================" << endl;
        cout << "          TEXT PROCESSOR MENU" << endl;
        cout << "=====================================" << endl;
        cout << "1. Load text from file" << endl;
        cout << "2. Save text to file" << endl;
        cout << "3. Display original text" << endl;
        cout << "4. Display sentences" << endl;
        cout << "5. Find most frequent letter in each sentence" << endl;
        cout << "0. Exit" << endl;
        cout << "Choose an action: ";

        int choice;
        cin >> choice;
        cin.ignore(); // Очищаем буфер после ввода числа

        switch (choice) {
            case 1: {
                string filename;
                cout << "Enter filename to load from: ";
                getline(cin, filename);
                processor.loadFromFile(filename);
                break;
            }
            case 2: {
                string filename;
                cout << "Enter filename to save to: ";
                getline(cin, filename);
                processor.saveToFile(filename);
                break;
            }
            case 3: {
                processor.displayText();
                break;
            }
            case 4: {
                processor.displaySentences();
                break;
            }
            case 5: {
                processor.findMostFrequentLetterInEachSentence();
                break;
            }
```

```
            case 0: {
                running = false;
                cout << "Exiting..." << endl;
                break;
            }
            default: {
                cout << "Invalid choice. Please try again." << endl;
            }
        }
    }

    return 0;
}
```

Файл **text.txt**, содержащий исходный текст для обработки:

```
text.txt U ×
C_codespace > 2 kurs > 3-rd senester > Programming Languages > Laboratory work > 4-th lab > Vlad > text.txt
This is a simple example text. It contains multiple sentences.
Each sentence should be analyzed carefully!
Can you see the pattern here?
Absolutely, every sentence matters.
Yes, indeed it does!
The quick brown fox jumps over the lazy dog. What a sight to behold!
Programming in C++ is powerful and flexible. It requires attention to detail.
Understanding algorithms is crucial. Data structures are important too!
Is this the last sentence?
No, this is another one. And this concludes the example.
```

Загрузка данных из файла **text.txt** и вывод в консоль:

```
=====================================
          TEXT PROCESSOR MENU
=====================================
1. Load text from file
2. Save text to file
3. Display original text
4. Display sentences
5. Find most frequent letter in each sentence
0. Exit
Choose an action: 1
Enter filename to load from: text.txt
Text loaded from text.txt
```

```
Choose an action: 3

--- Original Text ---
This is a simple example text. It contains multiple sentences. Each sentence should be analyzed carefully! Can you see the pattern here? Absolutely, eve
ry sentence matters. Yes, indeed it does! The quick brown fox jumps over the lazy dog. What a sight to behold! Programming in C++ is powerful and flexib
le. It requires attention to detail. Understanding algorithms is crucial. Data structures are important too! Is this the last sentence? No, this is anot
her one. And this concludes the example.
```

```
Choose an action: 4

--- Sentences ---
#1: This is a simple example text
#2:  It contains multiple sentences
#3:  Each sentence should be analyzed carefully
#4:  Can you see the pattern here
#5:  Absolutely, every sentence matters
#6:  Yes, indeed it does
#7:  The quick brown fox jumps over the lazy dog
#8:  What a sight to behold
#9:  Programming in C++ is powerful and flexible
#10:  It requires attention to detail
#11:  Understanding algorithms is crucial
#12:  Data structures are important too
#13:  Is this the last sentence
#14:  No, this is another one
#15:  And this concludes the example
```

Поиск и отображение буквы, встречающейся чаще всего в каждом предложении текста:

```
=====================================
          TEXT PROCESSOR MENU
=====================================
1. Load text from file
2. Save text to file
3. Display original text
4. Display sentences
5. Find most frequent letter in each sentence
0. Exit
Choose an action: 5
Sentence #1: E 4.
Sentence #2: E 4.
Sentence #3: E 7.
Sentence #4: E 6.
Sentence #5: E 7.
Sentence #6: E 4.
Sentence #7: O 4.
Sentence #8: H 3.
Sentence #9: I 4.
Sentence #10: T 6.
Sentence #11: I 4.
Sentence #12: T 6.
Sentence #13: E 4.
Sentence #14: N 3.
Sentence #15: E 4.
```

Сохранение текста в файл:

```
=====================================
          TEXT PROCESSOR MENU
=====================================
1. Load text from file
2. Save text to file
3. Display original text
4. Display sentences
5. Find most frequent letter in each sentence
0. Exit
Choose an action: 2
Enter filename to save to: text2.txt
Text saved to text2.txt
```

```
text2.txt U ×
C_codespace > 2 kurs > 3-rd senester > Programming Languages > Laboratory work > 4-th lab > Vlad > text2.txt
1   This is a simple example text. It contains multiple sentences. Each sentence should be analyzed carefully! Can you see the pattern
```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. **Охарактеризуйте класс vector, как происходит обращение к элементам?**

   Класс vector — это динамический массив. Доступ к элементам возможен через оператор [] или метод at(). Индексация начинается с нуля.

2. **Как происходит добавление/удаление элементов в классе vector?**

   Добавление: push_back() (в конец), insert() (в указанную позицию).
   Удаление: pop_back() (последний элемент), erase() (по итератору).

3. **При помощи какого метода можно указать количество элементов в классе vector?**

   С помощью метода size().

4. **При помощи какого метода можно изменить количество элементов в классе vector, как при этом будут происходить изменения?**

   Метод resize(n) изменяет размер: при увеличении добавляются элементы по умолчанию, при уменьшении — лишние удаляются.

5. **Охарактеризуйте класс array, как осуществляется его передача в функцию?**

   array — статический массив фиксированного размера. Передаётся в функцию по значению (копируется) или по константному указателю (const array&).

6. **Поясните, что из себя представляют контейнеры set/multiset, приведети плюсы и минусы использования set и multiset?**

   set и **multiset** — это ассоциативные контейнеры на основе сбалансированного двоичного дерева поиска. Они хранят элементы в отсортированном порядке и автоматически поддерживают эту сортировку при вставке и удалении. **set** хранит только уникальные элементы, **multiset** — с дубликатами.

   Плюсы: автоматическая сортировка, быстрота операций над элементами.

   Минусы: нельзя обращаться по индексу, вставка/удаление медленнее, чем в неупорядоченных контейнерах.

7. **Охарактеризуйте контейнер list. Приведите методы для добавления/удаления и обращения к элементам?**

   list — контейнер, реализующий двусвязный список. Каждый элемент хранит значение и указатели на соседние элементы. Обращение по индексу [] невозможно.

   Добавление: push_front(), push_back(), insert();
   Удаление: pop_front(), pop_back(), erase();
   Обращение: front(), back().

8. **Что такое итератор, для чего он нужен. Приведите пример (с применением синтаксиса) использования итераторов.**

Итератор — это обобщённый указатель на элемент контейнера, необходимый для его обхода.

Пример: for (auto it = vec.begin(); it != vec.end(); ++it) cout << *it << " "; it – итератор.

9. **Охарактеризуйте контейнер map. Как происходит работа с контейнером map?**

   map — ассоциативный контейнер «ключ–значение», элементы которого отсортированы по ключу в порядке возрастания.

   Доступ к элементу контейнера осуществляется по ключу: map[key] = value.

   Методы: find(), erase().

10. **Как проверить существование ключа в контейнере map?**

    С помощью метода count(). Если такого ключа нет в контейнере, будет возвращено значение 0.

    Пример: map_list.count(key) != 0.

11. **Поясните принцип и работу циклов foreach, какая форма записи у данных циклов?**

    Цикл **foreach** (диапазонный **for**) автоматически перебирает элементы контейнера.

    Синтаксис: for (auto& elem : container) { ... }. Работает с любым контейнером, имеющим begin() и end().

12. **Охарактеризуйте методы substr и replace класса string. Поясните принцип объединения строк.**

    substr(pos, len) — извлечение подстроки:

    pos – позиция начала подстроки; len – длина подстроки для извлечения.

    replace(pos, len, str) — замена подстроки:

    pos – позиция начала подстроки; len – длина подстроки для замены; str – строка, на которую заменяется подстрока.

13. **При помощи каких методов происходит добавление и вставка фрагмента в классе string. Перечислите методы подсчета количества символов строке string.**

    append(str); += – добавление в конец строки.

    insert(pos, str) – добавление в произвольное место строки.

    size(); **len() –** подсчёт количества элементов строки.

14. **Приведите синтаксис поиска первого вхождение другой строки (подстроки) в заданную строку.**

    С помощью метода **find(str)**: size_t pos = str.find("abc");

ВЫВОДЫ

Я получил практические навыки разработки и отладки программ по обработке строк с использованием стандартной библиотеки шаблонов (STL) на языке C++.